文件结构

- `cudabase.cuh`: 实现所需的核函数, 对启动核函数的封装, 以及对 CUBLAS 相关运算的封装.
- `functional.cuh`: 实现作业所要求的相关算子的正向和反向传播. 以调用 `cudabase.cuh` 的函数为主.
- `example.cu`: 测试程序, 构造了一个包含待测试算子的网络, 从 `stdin` 加载网络权值, 输出正向和反向传播的各处中间结果.
- `example.py`: 测试程序, 使用 `torch` 中的算子模拟 `example.cu` 的标准行为, 以检查正确性.
- `gen.cpp`: 权值生成器, 向 `stdout` 输出一组随机采样的浮点数, 作为网络权值.
- `chk.sh` 和 `cmp.py`: 前者不断使用 `gen.cpp` 生成测试数据, 交由 `example.cu` 和 `example.py` 运算, 并使用 `cmp.py` 比较. `cmp.py` 提取两个命令行参数对应文本文件中的每个浮点数, 当且仅当两个浮点数的相对或绝对误差不超过 10^{-4} 时, 认为二者相等. 当两个文件中的浮点数全部对应相等时, 认为两份文件相同.

辅助类和函数

```cpp
struct cublasHandleWrapper {
    cublasHandle_t handle;
    cublasHandleWrapper() { cublasCreate(&handle); }
    ~cublasHandleWrapper() { cublasDestroy(handle); }
    operator cublasHandle_t() const { return handle; }
} _handle;

struct curandGeneratorWrapper {
    curandGenerator_t generator;
    curandGeneratorWrapper() {
        curandCreateGenerator(&generator, CURAND_RNG_PSEUDO_DEFAULT);
        curandSetPseudoRandomGeneratorSeed(generator,
            (unsigned long long)std::chrono::steady_clock::now().time_since_epoch().count()
        );
    }
    ~curandGeneratorWrapper() { curandDestroyGenerator(generator); }
    operator curandGenerator_t() const { return generator; }
} _curandGen;
```

管理了 CUBLAS 和 CURAND 所需的对象.

```cpp
/**
 * @brief Compute y = alpha * op(A) * x + beta * y
 * @param A op(A) treated as a matrix of shape (M, N), ROW MAJOR
 * @param x treated as a vector of shape (N)
 * @param y treated as a vector of shape (M)
 */
void gemv(const float* A, const float* x, float* y,
          int M, int N, cublasOperation_t transA = CUBLAS_OP_N,
          float alpha = 1.0f, float beta = 1.0f);

/**
 * @brief Compute C = alpha * op(A) * op(B) + beta * C
```

```
 * @param A op(A) treated as a matrix of shape (M, K), ROW MAJOR
 * @param B op(B) treated as a matrix of shape (K, N), ROW MAJOR
 * @param C treated as a matrix of shape (M, N), ROW MAJOR
 */
void gemm(const float* A, const float* B, float* C, int M, int N, int K,
          cublasOperation_t transA = CUBLAS_OP_N, cublasOperation_t transB = CUBLAS_OP_N,
          float alpha = 1.0f, float beta = 1.0f);
```

二者封装了 CUBLAS 的矩阵-向量乘法和矩阵-矩阵乘法. 它们总是将输入矩阵视为 row-major 的矩阵, 这与 Tensor 的储存逻辑兼容, 但与 CUBLAS 的约定 (column-major) 相反. 为此, 二者将在函数内部适当转置矩阵或调整矩阵乘法顺序 (例如, $C = AB$ 可以调整为 $C^\intercal = B^\intercal A^\intercal$ 进行运算).

其余函数的逻辑较为简单; 对于接口复杂者, 大多有注释说明. 权值采样函数 `sample` 也在此实现. 但为了进行确定性的测试, 我们没有使用它对网络进行初始化. 详见正确性测试部分.

全连接层

```
void forward_fc(const float* X, float* Y, const float* W,
                const float* b, int N, int C_in, int C_out);

void backward_fc(const float* X, const float* Y, const float* W,
                 const float* b, int N, int C_in, int C_out, const float* dY,
                 float* dX, float* dW, float* db);
```

正向传播计算:

$$Y = X \times W^\intercal + b$$

反向传播计算:

$$\operatorname{grad} X = \operatorname{grad} Y \times W, \quad \operatorname{grad} W = (\operatorname{grad} Y)^\intercal \times X, \quad \operatorname{grad} b = \mathtt{colSum}(\operatorname{grad} Y).$$

正向计算时, 我们将 b broadcast 到 Y, 然后 $Y \leftarrow X \times W^\intercal + Y$. 反向传播时, 我们需要计算 Y 的梯度逐列 (即对每个 batch) 求和. 这两个行为都在 `cudabase.cuh` 中实现为 CUDA 函数.

卷积层

```
/**
 * @brief (zero-padding) & (square kernel with odd width) 2D convolution layer
 * @param X shape=(..., C_in, H, W)
 * @param Ker shape=(C_out, C_in, K, K)
 * @param b shape=(C_out)
 * @return shape=(..., C_out, H, W)
 */
void forward_conv2d(const float* X, float* Y, const float* Ker,
                    const float* b, int N, int C_in, int H, int W, int C_out, int K);

void backward_conv2d(const float* X, const float* Y, const float* Ker,
                     const float* b, int N, int C_in, int H, int W, int C_out, int K,
                     const float* dY, float* dX, float* dK, float* db);
```

按照课件中的思路实现. 不过正向传播为 $Y = W\hat{X}^\intercal$, 与课件实现略有差异 (或许这里课件有笔误). 两个关键函数 `img2col` 和 `col2img` 也在 `cudabase.cuh` 中实现.

池化层

```
void forward_maxpool(const float* X, float* Y, float* M, int N, int C, int H, int W, int K);

void backward_maxpool(const float* X, const float* Y, const float* M,
                      int N, int C, int H, int W, int K,
                      const float* dY, float* dX);
```

其中 `M` 保存了池化的 mask. 这个算子是容易实现的.

Softmax, Cross Entropy Loss

```
void forward_softmax(const float* X, float* Y, int N, int C);

void forward_celoss(const float* P, const int* L, int N, int C, float* loss);

void backward_softmax_celoss(const float* P, const int* L, int N, int C, float* dP);
```

按照课件的描述实现即可. 我们通过加上小量 `EPS = 1e-10f` 来防止对 0 取 $\log$.

正确性测试

测试方式已在文件结构中介绍. 我们测试的网络形如

```
# N = 2, Cin = 2, Cout = 3, H = 4, W = 4, K = 3, FCout = 5
# set in gen.cpp and can be modified

X0 = read_from_stdin()  # (N, C, H, W)
Kc = read_from_stdin()  # (Cout, Cin, K, K)
Bc = read_from_stdin()  # (Cout,)

X1 = conv2d(X0, Kc, Bc) # (N, Cout, H, W)
X2 = maxpool(X1, 2)     # (N, Cout, H//2, W//2)

Wl = read_from_stdin()  # (FCout, FCin)
Bl = read_from_stdin()  # (FCout,)

X3 = linear(X2, Wl, Bl) # (N, FCout)
X4 = softmax(X3)        # (N, FCout)
label = read_from_stdin(); # (N,) ranged in [0:FCout-1]
loss = cross_entropy_loss(X4, label) # scalar
```

由于

- 相同计算功能的不同实现方式可能引入不同的浮点误差;
- 对 Cross Entropy Loss 的小量 `EPS` 的选取方式

都可能导致我们的实现与 Pytorch 的标准结果产生非错误性差异, 我们采取了上述兼顾绝对误差和相对误差的比较方式. 此外, `EPS` 对 loss 值的影响 (在采样的测试数据而非真实神经网络运行环境中) 非常显著, 但不影响其他运算结果, 也不影响真实训练时的训练质量, 所以我们没有对二者最终的 loss 值进行对比.

编译

```
nvcc example.cu -o example -std=c++17 -lcublas -lcurand -DCHECK_SHAPE -O2
g++ gen.cpp -o gen -O2
```

此后运行 `./chk.sh` 即可. 正常运行时输出形如:

```
Files are equivalent within tolerance.
OK
Files are equivalent within tolerance.
OK
Files are equivalent within tolerance.
OK
...
```